um-game-playing-strength-of-mcts-variants

11th place solution

Rank	11
Team	Kansai-kaggler
Author	RYUSHI
Topic ID	549708
Version	Original

Team : kansai-kaggler (@tmhrkt , @ystsuji , @ryushisa)

First of all, I would like to express my sincere gratitude to the Kaggle staff and all the hosts of the competition for organizing such an amazing competition.

We would like to share our solution.

Overview

We adopted a strategy where each of our three members created individual single models, which we then stacked.

CV Strategy

We used StratifiedGroupKFold with the total number of games. We stratified by game total because we were concerned that the distribution might differ depending on the number of games, as mentioned in this discussion.

GameRulesetName and English Rules

Since the host clarified that the test data includes unseen GameRulesetName values, using TF-IDF features for GameRulesetName is risky.As for EnglishRules, we found the descriptions to be noisy, with variants such as Ludus Coriovalli.Therefore, we decided not to use GameRulesetName or EnglishRules as features.

Ryushi Part

Based on '[UM] Gradient Boosting Models Train&Infer'. This code was an extremely useful notebook that made it easy to experiment with LightGBM (lgb), XGBoost (xgb), and CatBoost (ctb). Ultimately, using only CatBoost yielded the best results, so I used that.

Features

- The best single model used only about 180 features.
- **Features that were removed**
  - Features with only one unique value
  - Features containing more than 95% missing values

- Frequency-based and `_components` columns. We did not remove all of them; we checked the distributions and kept those that seemed to have a reasonable amount of information. Details are as follows:
  - **Frequency features that were kept**
    `['DrawFrequency', 'StepDecisionFrequency', 'FromToDecisionEmptyFrequency', 'SetNextPlayerFrequency', 'RemoveEffectFrequency', 'MoveAgainFrequency', 'StepDecisionToEmptyFrequency']`
  - **Components features that were kept**
    `['NumStartComponentsBoard', 'NumStartComponentsHand', 'NumStartComponents', 'NumStartComponentsBoardPerPlayer', 'MarkerComponent']`
- Removed considering multicollinearity
  - For example, the following seven features had mutual correlations of over 90%. We kept only `'ScoreDifferenceVariance'` and removed the others:
    `['ScoreDifferenceVariance', 'ScoreDifferenceMedian', 'ScoreDifferenceMaximum', 'ScoreDifferenceChangeAverage', 'ScoreDifferenceMaxDecrease', 'ScoreDifferenceAverage', 'ScoreDifferenceMaxIncrease', 'ScoreDifferenceChangeLineBestFit']`
  - There were multiple columns that required such processing.
    By performing feature selection up to this point and parameter tuning, we were able to improve the Public LB score from 0.464 (baseline) to 0.422 (my single model).

- **Features that were added**
  - Length of LudRules
  - Created features from the `start`, `play`, and `end` sections of LudRules (details in the 2g part)
  - Extracted the number of occurrences of the words `'Draw'`, `'Win'`, or `'Loss'` from LudRules.
    - This was based on the hypothesis that there are patterns where the game itself is set to result in a draw besides timeout conditions.
      I had relatively few features in my part, but since other members had many features, we believe this contributed to diversity in the end.

CV/LB

	CV	Public LB	Private LB
exp057	0.4092	0.422	0.432
exp058	0.4087	0.422	0.432
exp059	0.4070	0.422	0.432
exp069	0.4082	0.424	0.432
exp071	0.4079	0.424	0.432

Ineffective Experiments

- **Neural Network Models**
  I conducted many experiments during the competition, but in the end, the experiments conducted in the middle were the ones that worked.
  In this competition, complex features and models actually resulted in decreased accuracy.
- **MultiRegression with CatBoost**
  Performed multi-regression in CatBoost to predict both `'utility_agent1'` and `'draw_ratio'`.

ktm part

- **Augmentation**: As other participants mentioned, swapping agent1 and agent2 with 1 - AdvantageP1, -utility_agent1 and added swapped flag column.When inferencing, TTA is applied.
- **Multi target prediction**: predicting utility_agent1 and draw_ratio with catboost MultiRMSE loss.
- **Data Generation**: We generated additional training data that does not appeared in the training data. we used publicly available rule on Ludii Portal and generated with colab cpu and vastai instance. (I had never thought I would rent a vast.ai instance just for the CPU.) The total generated data were about 30k~40k rows. This improved CV but LB did not change.
- target encoding

CV/LB

exp	CV	LB	private	target enc	Multi Target	additional data
exp084	0.4015	0.429	0.434	✓	✓	
exp088	0.4006	0.427	0.433	✓	✓	✓
exp094	0.4034	0.43	0.436	✓		✓
exp101	0.4016	0.429	0.435		✓	✓

2g Part

Based on MCTS Starter. The changes are as follows. We prepared multiple models by varying whether to apply the following:

- Added data by swapping `agent1` and `agent2`
- Predicted whether it would be a draw using binary classification. For predictions likely to be draws, set `utility_agent1` to 0.
- Divided `LudRules` into `start`, `play`, and `end` sections, and applied TF-IDF to each

- Created features from the `start`, `play`, and `end` sections of `LudRules` (for example):
  - `depth_nest`: Depth of nesting in the section (calculated using the number of `()` and `{}`)
  - `num_condition`: Number of `"if"` statements in the section
  - `chr_len`: Length of the section
  - `unique_start_pieces`: Number of `"place"` occurrences in the section
  - etc…

CV/LB

	CV	Public LB	Private LB
017-1	0.4092	unknown	unknown
018-1	0.4073	0.422	0.431
024-1	0.4070	0.422	0.431
025-1	0.4094	0.422	0.430

Stacking Part (Ryushi & ktm Part)

This part significantly improved our score. We had issues where the CV varied considerably across folds, and predictions changed even when only changing the model's seed between folds. To address this, we thought that stacking might be more effective than a simple ensemble. Also, since the scores varied significantly between folds, we optimized the weights (features) using Optuna for each fold. Our best model stacking is as follows:

CV	PublicLB	PrivateLB	Experiments used in the stacking training	Fold 0	Fold 0 CV	Fold 1	Fold 1 CV	Fold 2	Fold 2 CV	Fold 3	Fold 3 CV	Fold 4	Fold 4 CV
0.39122	0.4199	0.424	['ryushi_exp057', 'ryushi_exp058', 'ryushi_exp059', 'ryushi_exp069', 'ryushi_exp071', 'tsuji_017-1', 'tsuji_018-1', 'tsuji_024-1', 'tsuji_025-1', 'ktm084', 'ktm088', 'ktm094', 'ktm101', 'ktm084_draw',	['tsuji_025-1', 'ktm101', 'ktm088_draw']	0.3977	['ryushi_exp057', 'ryushi_exp058', 'ryushi_exp059', 'ryushi_exp069', 'tsuji_018-1', 'tsuji_024-1', 'tsuji_025-1', 'ktm084', 'ktm088',	0.36777	['ryushi_exp059', 'tsuji_018-1', 'ktm084_draw', 'ktm101_draw']	0.38142	['ryushi_exp057', 'tsuji_017-1', 'ktm084', 'ktm088', 'ktm084_draw',	0.40944	['tsuji_017-1', 'tsuji_024-1', 'ktm088', 'ktm094', 'ktm101', 'ktm084_draw', 'ktm088_draw']	0.39866

CV	Public LB	Private LB	Experiments used in the stacking training	Fold 0	Fold 0 CV	Fold 1	Fold 1 CV	Fold 2	Fold 2 CV	Fold 3	Fold 3 CV	Fold 4	Fold 4 CV
			'ktm088_draw', 'ktm101_draw']			'ktm094', 'ktm088_draw', 'ktm101_draw']				'ktm101_draw']			

Strategy for Selecting the Final Submission

We split the folds using total number of games, but we couldn't achieve a good correlation between CV and LB. Therefore, we ultimately selected the following two submissions:

- **CV+LB Best**: Selected the stacking model that had good CV and LB among stackings.
- **CV Best**: Selected the submission with the best CV among the subs where we optimized the weights using Nelder-Mead.
  (Note: We did not select the LB Best due to concerns about overfitting.)
  As a result, although the private score correlated with the public score, our selected CV+LB Best submission had the best private score .

Tips

Nelder-Mead

We performed ensemble by optimizing the weights of each submission using the Nelder-Mead method. When we performed the ensemble without the constraint that the sum of weights equals 1, the CV/LB scores improved (the sum of weights at that time was about 1.14). As mentioned in other people's solutions, it might have been important to multiply the predictions by a constant.